

Steam Games analyzation process

Intro

I recently completed Coursera's certification for Data Analyst and since I don't have a lot of practical experience in this field yet, I decided to do a project themed on PC video games- something I like to do in free time myself. My goal was to apply knowledge of what I recently learned in practice to my first research to see what makes video games successful. In this document I recorded whole data analysis cycle, steps I took to get to my results and explain my decisions on problems I encountered.

Why Steam games?

When I first started this project, I wanted to analyze successful video games in general. And when I started looking up data sources, I encountered big problems with several things:

- **Multiple data sources:**
 - Various consoles and some games on multiple consoles.
 - multiple major launchers with different games (EPIC, Steam, Riot Games, Blizzard...)
- **Data accuracy (constant changes to numbers sold):**
 - Pirated games from multiple sources,
 - Physical versions of games sold,
 - Constant purchases online
- **Publicity of data:**
 - Exact owner numbers kept private by launchers/studios,
 - Revenue numbers are private

There are several reasons I ended up using Steam's data as my source. The main reason is I started realizing scope of research I was attempting to do which with my experience and knowledge might be too much. At the end I decided to limit myself to only one reliable data source. I ended up using Steam platform because it has most data that I wanted to use publicly available and provides estimated numbers for some you can't get exact number for. Another is accuracy and relevance of data because you can access live data from server and get latest version of it.

Steam data- general information

About Steam API

The Steam API is a development interface provided by Valve. It allows external applications, websites and developers to securely communicate with Steam's servers and read public data from the platform. It's broken down into several endpoints-interfaces that serve different purposes (User data, game statistics and store data). Steam API operates as REST API; your application sends http request to specific Steam URL and Steam replies with data. Links I used to get data are:

<https://steamspy.com/api.php>

<https://store.steampowered.com/api/appdetails>

Interface acts as the 'bridge' that lets you pull data from Steam using a code and it returns that data by default in json format. In my case I used Python for which I learned basics during my education for Data Analyst certification. If you want to check out how I applied it, codes I used in this project to get data are posted in file directory file.

Data sources

I ended up using following data sources:

- **Kaggle** which is regularly updated (Version I downloaded on 3. 8. 26):
<https://www.kaggle.com/datasets/fronkongames/steam-games-dataset?resource=download>
- **Steam API database** using Python code to get specific data (links above)
- **Steam DB & Online data**, I used for my knowledge very advanced AI prompt, to scan and get data I needed (<https://steamdb.info/>). Added during research to speed up scraping data.

DATA PREPARATION

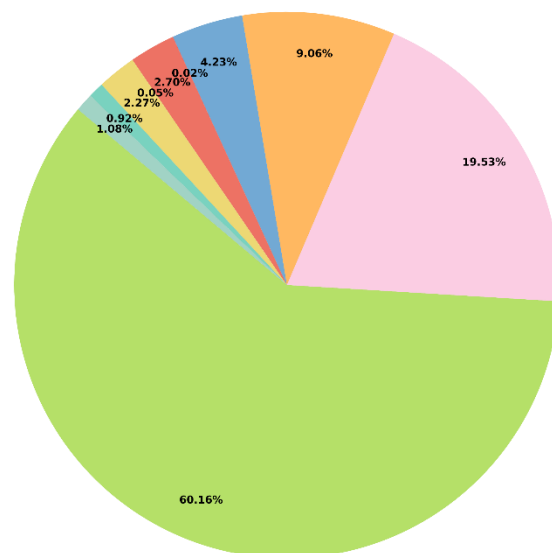
Once I had chosen my data sources, I first looked at what data (and in what shape) can I use for my research from Kaggle dataset. If you check Kaggle's dataset from link above, you will notice that it has 39 columns and staggering 125.848 rows. After a bit of research, I used dataset's column estimated owners to narrow down data I wanted to use. Explanation for estimated owners' calculation is further in document, and it uses **The Boxleitner Method** to get estimates.

Because getting accurate numbers of owners is impossible (no available public data and some game might get 100's of new owners as you read this which makes having exact data a nightmare), and because of how Boxleitner Method works, all games are put into estimated sales brackets which works in my favor because it took time to process this data and games might take longer time to get from one to other bracket.

Since my research focus was successfulness of Steam games, I could cut a lot of rows and columns out. What I ended up using is games that have more than 200.000 estimated sales which indicates that game was somewhat successful. What this meant for me in practice was that I ended up using 5362 rows of data which presents 4,27% of whole available dataset.

At this point I thought I had all the data I needed to go to next step but after discussing my project with friend who asked me, “What about newer games that didn’t achieve those numbers yet?”. He reminded me I didn’t include games that were released recently and didn’t accumulate over 200.000 owners yet which could impact research. For this reason, I added games that have over 50.000 owners and were released in 2026 and games that have over 100.000 sales and were released in 2025 or 2026. It turned out, I had to add 85 games so that brought me to 5447 rows of data which presents 4,34% of games on Steam. For easier interpretation of all this data, it is displayed on pie chart below accompanied with table for reference.

Owners Range	Number of Games	% of Total
1M+	1,353	1.08%
500k-1M	1,154	0.92%
200k-500k	2,855	2.27%
100k-200k	3,394	2.70%
100k+ (2025+)	59	0.05%
50k-100k	5,328	4.23%
50k+ (2026)	26	0.02%
20k-50k	11,396	9.06%
0-20k	75,705	60.16%
0-0	24,578	19.53%
Total	125,848	100.00%



As I mentioned above, I found that I can use Python to get data from Steam API directly which I wanted to try. I tried building code to get all games with estimated 200.000 or more sales myself. Code partly worked because I had trouble with responsiveness from server (I was denied access because I sent too many requests at once-server security denied access). At the end I remembered we live in 2026 and that AI is a thing. I used my original code and refined it with AI tools which implemented wait timer for sending requests and simplified some parts of code.

The Boxleitner Method

Because Valve considers exact amount of revenue from games and number of owners that own games strictly confidential datasets and community rely on estimated numbers. The work to get estimated number of sales was done by Data Scientists (Thank you!) who used **algorithmic estimation models** which work on The Boxleitner Method. It works on theory that there is a relatively stable ratio between the number of people who buy a game and the number of people who leave a review on it. And Steam publicly shows accurate amount of people who left a review. The Boxleitner Method works on following formula:

$$\text{Estimated Units Sold} = \text{Total Public Reviews} \times \text{The Boxleitner Number}$$

The Boxleitner Number is the multiplier for amount of people who bought a game and left a public review. Example: If a game had 1,000 reviews, you could assume it sold roughly 50,000 to 70,000 copies based on 50 to 70 multiplier range.

The multiplier shifted significantly over the years due to changes in player behavior and Steam's user interface:

- Historically (Pre-2019): The standard number was roughly 50 to 70.
- Recently (2020-now): The average multiplier is closer to 20 to 40.

Why the change in The Boxleitner's number?

Towards the end of 2019, Steam introduced a system-level pop-up that actively asked players to leave reviews for games. At that time percentage of buyers leaving reviews skyrocketed and lowered the multiplier because higher proportion of people left reviews.

Variables that skew Boxleitner Method

Method can produce skewed data because it's "order-of-magnitude" guide but it's the currently best thing we can use to estimate games sales numbers. The following factors can skew multiplier for individual games based on games characteristics:

- Free to play games do not apply to this method because someone can just add a game to their library and never play it
- Age of game highly affects because older games will have vastly higher multiplier attached to its lifetime reviews than a game released recently.
- Outside Sales don't apply to the method because it only tracks reviews written by people who bought the game directly on Steam.

As for me, research method was very helpful to filter out data I needed for project and it was also a solution to accuracy problem since people can buy and pirate games at this moment and exact data is impossible to get.

Genre data columns

Now that I have all my data, I have explored what else I could apply to my project and use as metric for research. What I decided to apply to my project are game genres and tags. Reason for adding genres is that I got idea to explore options if some genres are more successful than others and if niche tag exists that doesn't have a lot of games but it's successful. As for game tags, they are used to describe games visuals, themes, features, modes and moods. I was able to apply both because Steam keeps track of genres and tags voted by community for each game and data can be accessed by Steam API. Steam has 400+ existing tags and documented genres, and community can add new ones.

For easier terminology I will reference genre and tag as 'tag' in this document.

If you visit Steam's game's store page, you can see that there are tags displayed for each game. What you see are most voted ones in order. Steam shows top 5 most voted for each game (exact numbers aren't publicly known), but some games have a lot more tags than others (I encountered some games with 20+ tags). Because tags are voted for you can run into "troll" tags which might give you wrong information but for most part after checking data reliability it is quite accurate.

There are hundreds of tags on Steam, and most games are hard to describe with only one. I decided to subcategorize tags into multiple columns by features they describe and aspects they might provide for my project:

- **Genre:** Broader genre for game (contains most popular ones),
- **Subgenre:** More specific tag description of game,
- **Playing view:** Point of view game is played in,
- **Indie:** Yes/No as value (explained below)
- **Open World:** Yes/No as value (explained below)
- **Playing Options:** Describes singleplayer/multiplayer features for games
- **PvX Options:** Describes player vs environment/player features for games
- **DLC:** Yes/No/DLC as value to check if game has DLC/is DLC
- **Graphics:** Describes games graphics (2D, 3D, Pixel...)
- **Gameplay/Combat type:** Genres that describe gameplay/combat (shooter, puzzle...)

For all the above added columns I tried to use data from Steam's genre system and only searched for data online if it wasn't provided by Steam. I grouped features by similarity of what they describe. For example, playing view column has values: First-Person, Third-Person, Isometric, Top-Down, Side-view, Multiview (value for games that have multiple possible playing views that are listed) and Undefined (in case playing view can't be defined).

Indie and Open World values caused me a lot of trouble at beginning because they occurred very commonly and they don't really describe the game. To solve this problem, I decided to add them later as separate columns. Because of that, I could use more specific tags because they would often take place of subgenres which were intended for more specific genres and tags. Steam's data shows 96-99% of entries are Indie games, but they account for 48-58% of units sold on Steam which meant I could also explore aspects of what kind of Indie games succeed.

Game genre problems

At this point I encountered even more problems with genres because of huge amounts of similarities between some genres for which I would only need one value or part of tag's description. Easiest way to explain this is with examples and I will use few of them to describe my thinking behind decisions I made to exclude and adapt genre values.

First example is FPS (First-Person Shooter). Because I wanted to use shooter as genre in gameplay/combat column and First-Person as playing view, I decided to "split" this genre into two. So, you won't find this genre anywhere listed in this dataset, but you can still get to FPS games by combining filter where playing view is First-Person and combat is Shooter. I also applied this logic to Top-Down Shooter.

Second example also includes Shooter games. Because I wanted to avoid a lot of values for similar tags, I decided to combine all Shooter tags under same roof to simplify entry. This means whenever Shooter with 'something' appears (Arena Shooter, Looter Shooter, FPS...) it would just insert Shooter to decrease number of unique values for similar entries. Same logic was applied to Platformer games.

Third example is MMO (Massively Multiplayer Online) which I decided to stick into Playing options column because it includes a lot of other tags mechanics and it would make most of MMORPG entries identical in all columns. So, same as FPS I decided to split up MMORPG into RPG in Genre column and MMO in Playing options.

As I built vocabulary, I added more rules for tags which worked on principle of priority for multiple similar tags and workarounds for exceptions that I encountered. Because the list ended up quite long and would take a lot of pages to explain everything, I didn't include it in this document. But if it picked your interest, you could read more about them in [prompt_Steam.pdf](#) which contains all rules with detailed instructions for AI automation.

Huge workload problem

Since I would have to insert over 5000 rows of data, this task would take months to complete manually. I would also have to manually search for game tags and think of all rules and exceptions for every entry which would slow down all work even further. Here is where AI comes into play to automate this task for me. It took me some time and a bit of research, but I managed to make it work after several attempts.

Originally, I tried with excel file to have AI check each row for missing data, search it online and insert it into data cell directly which AI had a lot of problems to work with. Because of that I tried switching to different formats in case they're easier for AI to use. I ended up using .csv format for data entry.

So, I had 2 files to work with at the start. First one was the filtered Kaggle dataset (named 'games_Steam.csv') that consisted of columns AppID, Name, Release date, estimated owners, Review Score (%), Positive Reviews, Negative Reviews and Price of games I researched. All these columns were already filled with data. Then it also had empty columns that are all identical to ones in second .csv file.

Second file had all of tags columns (listed and described above at game genre problems) with tags listed in columns where they're allowed to use as entry. File worked as 'vocabulary' for AI as reference to "Where can it insert specific values".

Using these 2 files I used AI to create a prompt which it used as reference at start of every session for rules and conditions when it filled out the missing tag data. Whole prompt worked in a way it took batch of games and requested tags column data from API for those games. Back it received all genres for each game that were by default ordered by votes with most voted ones listed first. Then it checked the whole existing tags list for game and then for each column where tag was valid entry, inserted only most voted one for that column. If no value could be filled, it searched for viable value first in game description and then online which happened mostly with playing view column because a lot of games didn't have it listed on their tag list.

Soon after it started working automatically, it ran into problems with token limits. Because I used free version of Claude, it hit limit at some point just by reading a file. Solution to it was to make a log file (progress_log.md file) into which it wrote which games and what values it used and which games were done. It went in order, so it always continued from entry where it finished last session. After few sessions I optimized log file entry so it used less tokens for this task and got more games done in session (you will notice change in log file entry if you check the file).

Although I managed to automate this task for AI to do it, it still took **about 1 month** to fill out data from vocabulary! But you must consider that I used free version of Claude which limited my tokens (session and weekly limit). I also decided to use one AI tool for this heavy lifting because when I tried doing it with multiple tools at once, I was getting inconsistent results for task at hand.

Because it was taking so long and automation was 'slow at work' at some aspects, but still much faster than if I did it manually, I started to search for ways to help fill out the missing data faster. After brief research of what I could access over API, I ended up using python scripts to manually request data columns. It ended up working for some columns (Indie, Open World, partially tags pull, DLC and Graphics) to just request data with python to API and insert it to correct row with appID as reference. It worked for those entries because Indie, Open World and DLC were only Yes/No (TRUE/FALSE) values and it checked for specific values if they exist and it returned value. As for Graphics tag entry (2D, 3D, Pixel Graphics...) it checks only their existence and write it if it existed and those that were left were searched for online.

You can try to get these tags for any Steam game, but you'll need a .csv/.json file in same directory since it will be used to write data into it. Code must include column reference that exists in Steam database (AppID or Game Name). It's recommended it also includes prepared empty columns to write data into.

Python code could potentially be upgraded and check existing tags and fill all missing columns by itself, but I didn't attempt to do that simply because I lack knowledge to do so.